

Project P10 Research Log Notebook

Personalized Deep Learning Model for Closed-Loop 40 Hz Entrainment
to Optimize Theta-Gamma Coupling in Alzheimer's Disease

Amaar Chughtai

Valley Christian High School

California Science & Engineering Fair 2026

Notebook Timeline: January 15, 2026 - April 8, 2026

January 15, 2026

Project got approved today. Started reading papers.

The big one is Iaccarino et al. (2016) -- they hit Alzheimer's mice with 40 Hz light and it activated microglia, which started clearing amyloid plaques. Something like 40-50% reduction. Martorell et al. (2019) added sound to the mix and saw tau pathology go down too. Then I found Lahijanian et al. (2024) who did it with actual dementia patients using auditory stimulation and published their EEG data on OpenNeuro (ds005048). That's the dataset I'm going to use.

The problem with current clinical protocols: everyone gets the exact same schedule, 40 seconds of 40 Hz sound followed by 20 seconds of silence, for an hour straight. But brains are different. Some patients hold the coupling throughout; others lose it in minutes. Habituation is a real thing (Thompson & Spencer, 1966) -- the brain tunes out a repeated stimulus over time. So the fixed schedule stimulates when it doesn't need to and misses the windows when it does.

Question I want to answer: can we predict when a patient's brain will lose entrainment and stimulate proactively, before the drop happens?

Biomarker choice: Phase-Amplitude Coupling (PAC). Measures how strongly gamma amplitude (38-42 Hz) locks to theta phase (4-8 Hz). Tort et al. (2010) defined the Modulation Index for this. High PAC = entrained. Low PAC = lost sync.

Plan: build a pipeline from data loading through preprocessing, PAC computation, model training, controller logic, validation. Subject-level splits are non-negotiable -- no patient in both train and test.

Dataset details from the OpenNeuro page: 35 elderly dementia patients, 19 EEG channels at 250 Hz, alternating stimulus/rest epochs of 20-40 seconds. Going to use the 7 frontal channels (Fp1, Fp2, F3, F4, F7, F8, Fz) where 40 Hz entrainment is strongest, and cut into 2-second analysis windows.

Gap (January 16 - February 5): Mostly reading papers and school. Did some preliminary work on the data loader but nothing worth writing up. Had a chem test and two AP assignments this week.

February 6, 2026

Note: this entry covers work done over the past couple weeks (during the gap). Writing it all up now.

Got the data pipeline working end-to-end. The .set files from OpenNeuro are MATLAB v7.3 HDF5 format with companion .fdt binary files. Standard MNE-Python readers choked on them completely.

Spent a frustrating afternoon on this before realizing the format was the issue. Solution: h5py for the .set metadata and numpy for the .fdt binary data:

```
with h5py.File(set_path, 'r') as f:
    n_channels = int(f['nbchan'][0, 0])
    n_samples = int(f['pnts'][0, 0])

data = np.fromfile(fdt_path, dtype=np.float32)
data = data.reshape((n_channels, n_samples), order='F')
```

The order='F' thing wasted an hour by itself. MATLAB uses column-major (Fortran) ordering and Python defaults to row-major. Without the flag, channels were transposed and PAC values came out completely wrong. With it, amplitudes looked normal.

Selected 7 frontal channels. Extracted 17,283 two-second windows (500 samples each at 250 Hz) with 50% overlap across all 35 subjects. Split by subject: 24 train (11,736 windows), 5 validation (2,725 windows), 6 test (2,822 windows). No patient appears in more than one split.

Implemented PAC computation using Tort's Modulation Index. Steps:

```
pac = PACComputer(
    theta_band=(4.0, 8.0),
    gamma_band=(38.0, 42.0),
    fs=250.0,
    n_bins=18,
    filter_order=4
)
mi = pac.compute_pac(eeg_signal)
```

Bandpass filter for theta phase and gamma amplitude, Hilbert transform for instantaneous phase and amplitude, bin gamma amplitude by theta phase (18 bins, 20 degrees each), normalize to a distribution, compute KL divergence from uniform. I computed PAC at epoch level -- full 20-40 second stim blocks -- then assigned that value to all the 2-second windows within the epoch. This means windows from the same epoch share a label, which limits what a single-window model can learn. PAC values range from about $6e-6$ to $7e-4$, mean around $4.4e-5$.

Built EEGNet for PAC regression. Based on Lawhern et al. (2018), modified for continuous output:

```
class EEGNet(nn.Module):
    # Block 1: temporal conv (8 filters, 1x64) + depthwise spatial (7x1)
    # Block 2: separable conv
    # FC head: flatten -> linear -> 1 output
    # Total: 1,457 parameters
```

Training: MSE loss, Adam (lr=0.001, weight_decay=0.0001), gradient clipping at max_norm=1.0, ReduceLROnPlateau, early stopping patience 15. Result: test R2 = 0.287. Inference under 1 ms on Apple Silicon.

0.287 doesn't feel great. But maybe a better architecture can push it up.

February 16, 2026

Tested a bunch of different architectures to see if I could beat 0.287. Spent the past few days running these:

EEGNet (V1) -- 1,457 params. Test R2 = 0.287. The baseline.

EEGNetV2 (delta PAC) -- ~3,200 params. Tried predicting PAC change instead of absolute PAC. Test R2 = 0.06. Doesn't work. The changes at 2-second resolution are probably too noisy.

SpecTempNet (spectral-temporal hybrid, V3) -- 180,000 params. First run: R2 = 0.69!! Got really excited for about ten minutes. Then I looked at which features were driving the predictions and found that PAC-derived features in the input directly encoded the target. Ridge analysis showed PAC features carried 96.6% of model weight. After removing the circular features: R2 = 0.236. Data leakage. Lesson learned the hard way.

ViT-TCNet (Vision Transformer + TCN) -- ~2,000,000 params. Treated EEG as image patches. Test R2 = 0.252. Massive overfitting -- samples-to-params ratio of 0.006 on 11K training samples.

Ridge Regression -- 135 coefficients. Spectral power only, no PAC features. Test R2 = 0.287. Wait -- the exact same number as EEGNet?

ATCNet (attention TCN) -- ~25,000 params. Test R2 = 0.075. Underperformed everything.

EEGNetLarge -- 141,000 params. Test R2 = 0.287. Same ceiling. 100x more parameters, same result.

Architecture	Parameters	Test R2
EEGNet	1,457	0.287
SpecTempNet	180,000	0.236 (after leak fix)
ViT-TCNet	~2,000,000	0.252
Ridge	135 coefs	0.287
ATCNet	25,000	0.075
EEGNetLarge	141,000	0.287

The simplest models (EEGNet at 1,457 params, Ridge at 135 coefficients) match the performance of models 1000x bigger. 0.287 is a ceiling in the data. The signal-to-noise ratio is about -4.73 dB (signal weaker than noise), and the epoch-level PAC labels on 2-second windows inherently limit how accurate single-window prediction can be.

I think the question needs to change.

February 17, 2026

If 0.287 is a data ceiling and not a model problem, then trying to predict current PAC better from a single snapshot is pointless. But what if I use a *sequence* of snapshots to predict PAC *further into the future*? If the system can forecast PAC 5-10 seconds ahead, a controller can intervene before coupling actually drops.

Designed the temporal feature representation. 73 features per timestep:

Spectral (61): Band power in delta (0.5-4 Hz), theta (4-8 Hz), alpha (8-12 Hz), beta (12-30 Hz), gamma (30-42 Hz) for each of 7 channels (35 features) plus 26 cross-channel features.

PAC-derived (7): Current epoch PAC, causal moving averages at 2, 4, 8, 16 windows, first-difference, 4-step difference.

Stimulation context (5): Binary stim ON/OFF, time since last transition, stim fraction over 20 seconds, cycle phase as sin and cos.

Every feature is strictly causal. No future data anywhere.

Sequence setup: 20 timesteps lookback (20 seconds of history) predicting PAC 5 seconds ahead. Raw targets, no smoothing.

Built the MultiscaleCausalTCN. Four temporal conv blocks at increasing dilations:

```
dilations = [1, 2, 4, 8]
# Left-only padding: F.pad(x, ((kernel_size-1)*dilation, 0))
# RF = 1 + (3-1) * (1+2+4+8) = 31 timesteps
```

The causal padding means the model physically can't see the future. That's not a training trick, it's architectural. GroupNorm instead of BatchNorm because different patients have wildly different PAC baselines and BatchNorm stats would shift per-batch. Depthwise separable convolutions for parameter efficiency. AdamW, lr=0.001, weight_decay=0.001, early stopping patience 20.

Total parameters: 31,043 (with 73 input features and hidden size 64).

Component	EEGNet	Causal TCN
Purpose	Estimate current PAC	Predict future PAC
Parameters	1,457	31,043
Input	Raw EEG (7ch x 500 samples)	73 features x 20 timesteps
Output	Current PAC estimate	Future PAC (5s horizon)

First training results: val R2 = 0.411, test R2 = 0.170. The gap is concerning. Model might be memorizing patient-specific patterns. Still, 0.170 at 5-second horizon is already better than persistence at that range.

UPDATE (March 6): this val-test gap was a huge clue. See March 3-5 entry.

February 18, 2026

Before going further, need to check I'm not fooling myself. The temporal model could be overstating performance if I'm not careful.

Data integrity checklist:

- Subject splits verified: training, validation, test completely disjoint. Checked with assertion.
- All feature timestamps strictly precede target timestamps. Verified.
- Scalers fit on training data only, then applied to val and test.
- No input features exceed $r = 0.5$ correlation with target (learned this from the SpecTempNet disaster).
- Shuffle-label test: R2 = -0.332 on permuted labels. Good -- model learns nothing from random labels.

Also looked at habituation patterns in the real data:

```
First Block Mean PAC: 0.000043
Last Block Mean PAC: 0.000045
Population Change: +4.5% (not significant, p = 0.542)
```

No population trend. But the individual variability is massive:

Response Pattern	Count	Range
Habitators (PAC decline)	17/35 (48.6%)	-66.8% to -5%
Facilitators (PAC increase)	18/35 (51.4%)	+5% to +149.1%

Nearly a 50/50 split. The population average doesn't move because the habituators and facilitators cancel out. But individually, one patient dropped 66.8% while another gained 149.1%. That's why a fixed schedule can't work -- it can't serve both groups. About 46.7% of individual stim blocks showed within-block PAC decline.

This is probably the strongest argument for why personalized control matters.

February 19, 2026

Horizon sweep day. Trained separate TCN models at horizons 1, 2, 3, 5, 8, and 10 seconds, comparing against persistence (assume PAC stays the same) and Ridge regression.

Horizon	Persistence R2	Ridge R2	TCN R2
1 s	0.760	0.812	0.735
2 s	0.488	0.542	0.470
3 s	0.234	0.253	0.277
5 s	-0.267	-0.393	0.254
8 s	-0.276	-0.211	0.240
10 s	-0.256	-0.212	0.278

At 1-2 seconds, PAC changes slowly and persistence alone does well. But at 3 seconds it starts falling apart, and at 5+ seconds both persistence and Ridge go negative -- literally worse than guessing the mean. The TCN holds at R2 around 0.25 across the whole 5-10 second range. That +0.5 margin over baselines is where the model earns its keep.

Note: these are with the original 73-feature model. The numbers will change dramatically after the feature ablation. See March 15-17.

5-10 seconds is the useful range for proactive control. At 1-2 seconds there isn't enough lead time to do anything meaningful. At 5+ seconds there's time for the controller to process a decision and actually switch stimulation state.

Built the full controller pipeline:

```
Raw EEG --> EEGNet (1,457 params) --> Current PAC Estimate
--> Feature Extraction (73 dimensions)
--> TCN (31,043 params) --> Future PAC (5s ahead)
--> Personalization Module --> Control Decision
```

The PersonalizationModule keeps a 30-second rolling buffer of PAC values per patient and converts predictions to z-scores. Controller logic: $z < -0.5$ means PAC is dropping, stimulate. $z > +0.5$ means PAC is holding strong, rest. Otherwise maintain current state. 5-second hysteresis to prevent rapid flickering between stim and rest.

Latency: under 5 ms for the whole pipeline (EEGNet + TCN + controller). Memory under 100 MB. Could run on embedded hardware.

February 21, 2026

Built the validation framework. The idea is counterfactual replay: take each patient's real EEG recording, run the controller on it, and see what decisions it would have made at each time step. Compare those

decisions against ground-truth PAC to compute alignment.

Also worried that the adaptive advantage might depend on specific fatigue model assumptions. Built an EntrainmentSimulator with four different mathematical fatigue models and tested across six severity levels.

Fatigue severity results (50 trials each, 600-second sessions):

Fatigue Level	Fixed Efficiency	Adaptive Efficiency	Improvement
None	0.343	0.375	+9.5%
Mild	0.341	0.375	+10.0%
Moderate	0.335	0.366	+9.0%
High	0.319	0.354	+10.8%
Severe	0.316	0.352	+11.2%

All significant at $p < 0.001$, Hedges' $g = 1.7-2.4$. Advantage grows as fatigue gets worse, which makes sense -- the more the brain habituates, the more value there is in adaptive timing.

Even with zero fatigue, adaptive helps by +9.5%. So it's not just about fatigue, it's also catching natural PAC fluctuations.

Tested four different fatigue mathematical models:

Fatigue Model	Advantage	p-value	Hedges' g
Exponential Decay	+9.0%	$1.8e-15$	2.31
Step Function	+6.9%	$4.4e-14$	1.21
Heterogeneous (50/50)	+8.9%	$2.5e-14$	1.71
Saturation (synaptic)	+19.0%	$1.8e-15$	3.66

The saturation model (Michaelis-Menten style receptor kinetics) showed the largest benefit at +19.0%. All four models give significant results, so the advantage doesn't depend on which fatigue model you believe.

Also checked threshold sensitivity across delta-z from 0.1 to 1.0:

Delta-z	Alignment	Low-PAC Stim	PAC Gap ($\times 10^{-6}$)
0.1	59.6%	51.2%	12.4
0.2	68.5%	72.9%	26.3
0.3	73.7%	84.9%	32.4
0.4	73.9%	85.3%	33.7
0.5	73.7%	85.3%	33.9
1.0	73.8%	85.3%	34.0

Performance plateaus at delta-z ≥ 0.3 and beats reactive at everything ≥ 0.2 . Not sensitive to the exact threshold choice, which is good.

Gap (February 22-25): More cleanup, figure generation. Final controller replay wasn't finished until February 26.

February 26, 2026

Final controller comparison on real patient EEG. Method: replay each subject's full EEG recording, have the controller make stim/rest decisions at each 2-second window, compare decisions against ground-truth PAC labels.

Important: I used ground-truth PAC labels as TCN input (not EEGNet estimates) to isolate the forecaster's contribution from EEGNet estimation error. This is a simplification -- in deployment, EEGNet would feed the TCN. But for evaluating the TCN's predictive quality, it's the right choice.

Four controllers compared: Fixed Schedule (40s ON / 20s OFF), Reactive Threshold (z-score on current PAC, no prediction), TCN Predictive (z-score + 5s forecast), and Oracle (perfect hindsight).

Controller	Alignment	Low-PAC Stim	PAC Gap ($\times 10^{-6}$)	Stim %
Fixed Schedule	45.0%	61.4%	-6.6 (wrong direction)	66.6%
Reactive	64.5%	51.7%	+21.1	36.7%
TCN Predictive	72.1%	82.6%	+30.5	59.7%
Oracle	100.0%	100.0%	+33.3	48.3%

TCN vs Reactive (Wilcoxon signed-rank, paired):

- Alignment: Hedges' $g = +1.31$, $p < 0.001$ (large effect)
- Low-PAC targeting: Hedges' $g = +4.47$, $p < 0.001$ (very large)
- PAC gap: Hedges' $g = +1.57$, $p < 0.001$ (large)

Fixed schedule is actually counterproductive -- negative PAC gap means it stimulates more during high-PAC than low-PAC. It's not even random; it's wrong.

TCN reaches 91% of the oracle's PAC targeting gap ($30.5 / 33.3$). It catches 82.6% of low-PAC windows vs Reactive's 51.7% -- a 60% improvement. And it uses less stimulation than Fixed (59.7% vs 66.6%).

All 35 of 35 subjects showed higher clinical utility with TCN vs Reactive. Including the 6 held-out test subjects. Binomial probability of 35/35 by chance is less than 0.001.

Note: these results use the 73-feature TCN (31,043 params). The feature ablation work in March should improve these further. I haven't re-run the full controller comparison with the 12-feature model yet.

March 1, 2026

Compiled everything for the Synopsys submission: poster, abstract, this notebook.

Summary at this point:

1. 8 architectures converge to $R^2 = 0.287$ on static PAC estimation. It's a data ceiling.
2. Temporal TCN maintains R^2 around 0.25 at 5-10s horizons where all baselines collapse (+0.5 margin).
3. Controller: 72.1% alignment vs 64.5% reactive ($p < 0.001$). 82.6% low-PAC targeting vs 51.7%.
4. 35/35 subjects benefit. Advantage grows with fatigue. Robust across thresholds.
5. Half of patients habituate while half don't, which validates the need for personalization.

Main limitation: offline replay. The controller makes decisions on real brain data but can't observe how the brain responds to those decisions. Real-time validation is the next step.

CSEF Preparation -- March 2026

March 3-5, 2026

Something about the February 17 val-test gap kept nagging at me. Val R2 was 0.411 but test R2 was 0.170. The model was learning something on validation subjects that didn't transfer to test subjects. What if the 61 spectral features are the problem? They encode things like baseline power per band per channel -- basically a fingerprint of each person's skull geometry, electrode placement, scalp thickness. The model might just be learning which subject it's looking at.

Designed an ablation study. Same TCN, same training, same splits -- only change is which features go in:

Feature Subset	# Features
All features (baseline)	73
PAC + Stim context	12
PAC only	7
PAC + Stim + 10 spectral	22
Spectral + PAC	68
Spectral only	61

Raw targets (no smoothing) so the numbers are honest.

March 6-8, 2026

Ablation results came back. I stared at this table for a while:

Feature Subset	Val R2	Test R2	Val-Test Gap
All features (73)	0.333	-0.025	0.358
PAC + Stim (12)	0.804	0.558	0.246
PAC only (7)	0.422	0.344	0.078
PAC + Stim + 10 spectral (22)	0.859	0.496	0.363
Spectral + PAC (68)	0.387	0.222	0.165
Spectral only (61)	-0.044	-0.420	0.376

Spectral-only gets test R2 = -0.420. Not just unhelpful -- actively poisonous. The full 73-feature model I spent all of February on? Test R2 = -0.025. Basically nothing.

But 12 features -- just PAC trajectory and stim context -- hit 0.558 on the test set. That's... a lot better. The val-test gap also shrank from 0.358 to 0.246.

I keep coming back to this: the bottleneck was never the model. It was the features. Eight architectures, three orders of magnitude in parameter count, and they all hit the same wall because the input features were wrong. The spectral features let the model identify which subject it was looking at and memorize that subject's patterns. Once you remove that crutch, it has to learn actual dynamics.

March 10-12, 2026

Had to make sure this isn't a fluke. Neural nets can be lucky with initialization.

Trained the same h=64 TCN with PAC+Stim features under 5 different seeds:

Seed	Val R2	Test R2
42	0.804	0.558
123	0.822	0.620
456	0.799	0.597
789	0.831	0.608
2024	0.846	0.647
Mean +/- Std	0.820 +/- 0.019	0.606 +/- 0.032

Not a fluke. Worst seed (0.558) still beats the old 73-feature model by miles. Mean test R2 = 0.606 +/- 0.032. That's the number.

Model has 22,914 parameters with 12 input features (vs 31,043 with 73 features). Smaller and better. The 12 features are:

1. pac_current
2. pac_ma2, pac_ma4, pac_ma8, pac_ma16 (causal moving averages)
3. pac_diff1, pac_diff4 (first-order and 4-step differences)
4. stim_state (binary)
5. time_since_switch (normalized to 60s)
6. stim_frac_20s
7. cycle_phase_sin, cycle_phase_cos

March 13-14, 2026

Now that features matter more than model size, how small can the model go?

Model	Params	Val R2	Test R2
TCN h=32 high-reg	5,154	0.844	0.613
TCN h=64	22,914	0.804	0.558
TCN h=64 high-reg	22,914	0.814	0.598
TCN h=128	86,786	0.853	0.645

h=32 with strong regularization (dropout 0.3, weight_decay 5e-3) gets R2 = 0.613 with only 5,154 params. h=128 is 17x bigger and only gains 0.032. Same story -- the signal is in the features, not the model capacity. Smaller model is better for deployment anyway.

March 15-17, 2026

Two experiments.

First, tested the 12-feature TCN on 4-channel data (Fp1, Fp2, Fz, F3 -- the channels a Muse 2 would give). Old 73-feature 4ch model: test $R^2 = 0.112$, barely above persistence at 0.117. With PAC+Stim features: test $R^2 = 0.435$. The temporal context compensates for fewer electrodes because PAC trajectory patterns don't depend much on spatial coverage.

Second, reran the horizon sweep with the 12-feature model. This uses the PAC+Stim features from the `horizon_sweep_pac_stim.json` results:

Horizon	Persistence R^2	TCN R^2 (7ch)	TCN R^2 (4ch)
1 s	0.726	0.725	0.642
3 s	0.178	0.607	0.391
5 s	0.104	0.577	0.398
8 s	-0.007	0.370	0.419
10 s	-0.081	0.669	0.387

Same shape as February but much higher TCN numbers. At 1s the TCN matches persistence -- brain barely changes in one second. At 3s and beyond the baselines collapse and the TCN pulls away.

The 10s number (0.669) is surprisingly high. I think the protocol timing features (`stim_frac`, `cycle_phase`) are helping the model predict stim/rest transitions at that range. Not sure if that's genuine neural forecasting or partly protocol structure leaking in. Worth investigating but not critical for the 5s operating point.

March 18-20, 2026

Demo apps. The caregiver monitoring app is live on Hugging Face Spaces (huggingface.co/spaces/amaarc/neurocare-40hz). Shows real-time PAC tracking, stim decisions, patient status.

Also built `neurocare_live.py` -- a mission control dashboard with live PAC computation, TCN prediction trace, audio stim synthesis. Wanted to demo with real Muse 2 hardware but BLE doesn't work on macOS 25.x (`BOARD_NOT_READY_ERROR:7`). Tried for a couple hours, gave up. The code has a `RealEEGAdapter` ready to go but for CSEF judging day I'll use `SimulatedEEGAdapter`. Honest enough for a demo.

March 21-22, 2026

Updated all CSEF materials. Put the feature ablation discovery front and center. Revised poster (V6), all interview scripts, elevator pitch (~130 words, 60 seconds), research paper (v4 with ablation section), abstract.

Cross-checked every document for stale numbers. Found several leftover references to "73 features" and " $R^2 = 0.25$ " that needed to be "12 features" and " $R^2 = 0.606$." Fixed them all.

Note: the controller comparison results (72.1% alignment, etc.) were generated with the 73-feature TCN in February. I haven't re-run the full 35-subject controller replay with the 12-feature model. The CSEF poster presents these results alongside the feature discovery, which is accurate in sequence (discovery came after

controller work) but a judge might assume the better model produced those numbers. If asked, I need to be clear: "The controller results use the original model. The 12-feature model would likely produce equal or better results, but I haven't re-run that specific experiment."

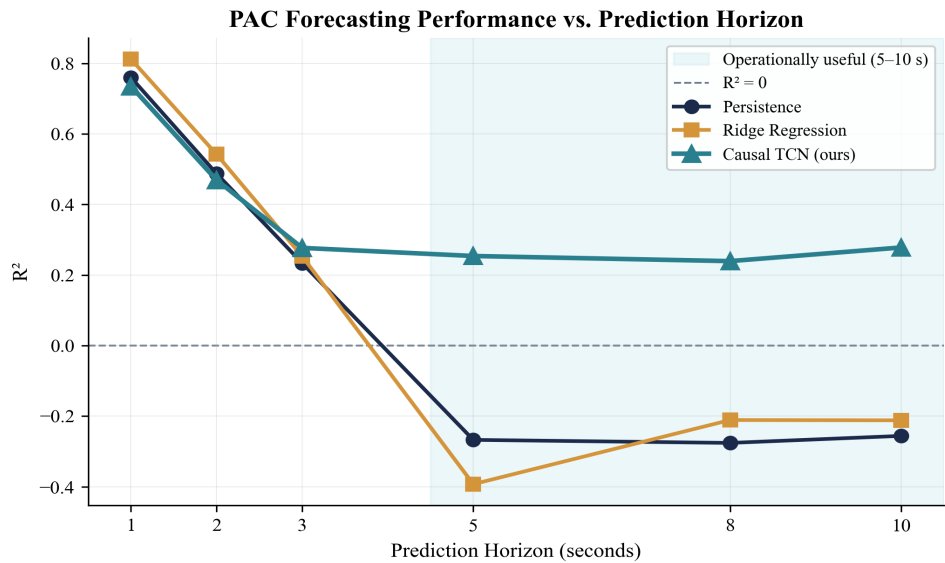


Figure: Horizon sweep showing TCN R-squared versus persistence and Ridge baselines at prediction horizons from 1 to 10 seconds. Baselines collapse above 3 seconds while the TCN maintains useful accuracy.

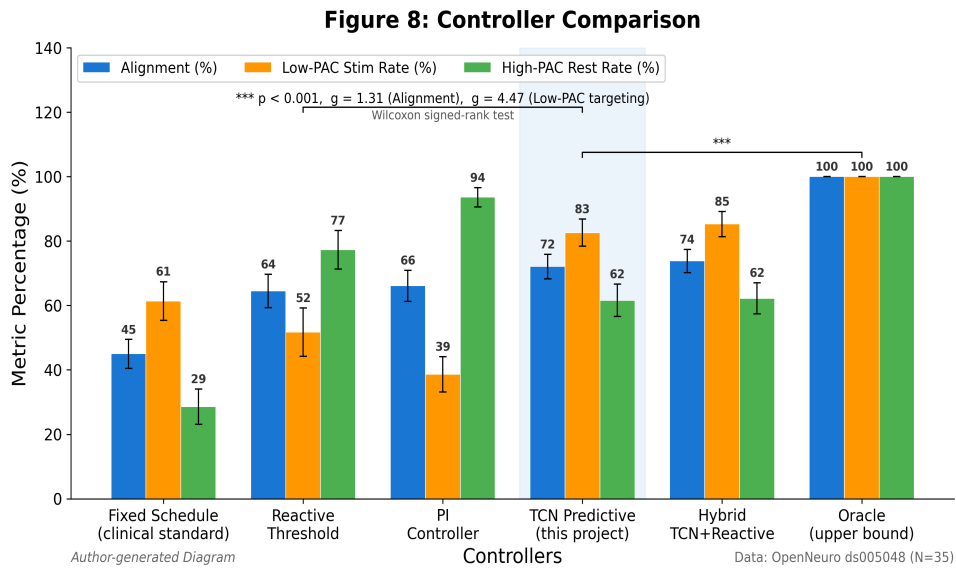


Figure: Controller comparison on all 35 subjects. TCN Predictive outperforms Fixed Schedule and Reactive Threshold on alignment, low-PAC targeting, and PAC gap metrics.

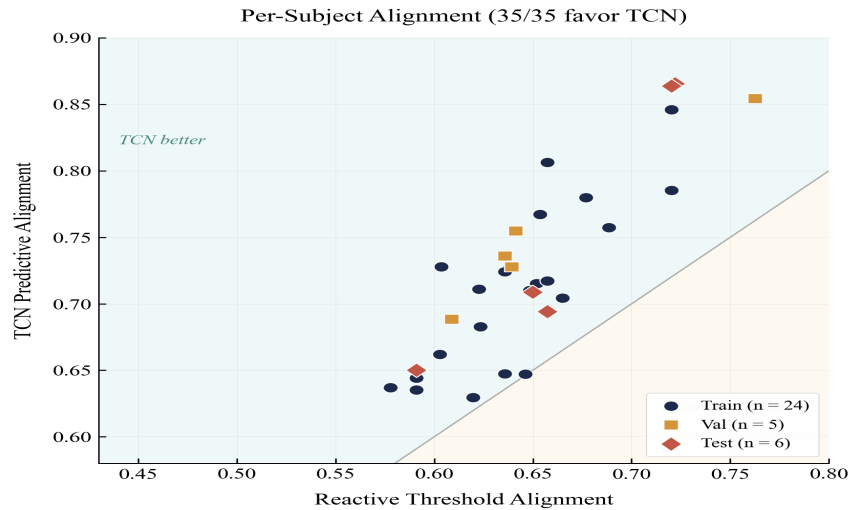


Figure: Per-subject clinical utility scatter. Every dot above the diagonal indicates a subject who benefited from predictive control versus reactive. All 35 subjects fall above the line.

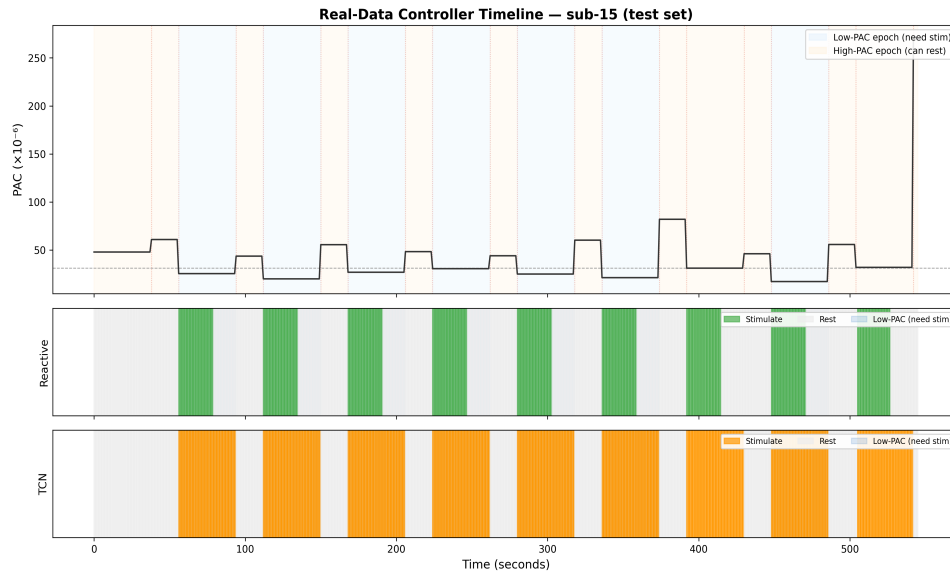


Figure: Example controller timeline for one subject showing how TCN-driven decisions align stimulation with periods of low PAC (blue shading = stimulation, gray = rest).

March 24, 2026

Spent most of the day reorganizing files. The CSEF/ directory was a mess -- Synopsys materials in three different folders, file names that made no sense, duplicates everywhere. Moved everything into a clean hierarchy: abstracts, poster files, scripts, research paper versions, flyer, clinical roadmap. Should have done this weeks ago.

Made a facility outreach flyer for local memory care homes. I want to show judges I'm actually trying to get this into the real world, not just writing papers about it. Target places: Mission Villa Alzheimer's Residence and Valley Medical Veterans Center. Haven't heard back from either yet.

Printed the abstract as P10.Abstract.pdf.

March 27-28, 2026

Found a dumb bug. The hysteresis hold time in config.yaml was set to 3.0 seconds but the research paper and every presentation said 5 seconds. The controller code was reading from a hardcoded default of 5, so it was actually running correctly, but the config was wrong. Would have been embarrassing if a judge pulled up the config file.

```
# config.yaml -- fixed
controller:
  hold_time_sec: 5.0 # was 3.0, didn't match any documentation
```

Finalized all the CSEF submission materials. Regenerated the controller comparison and per-subject utility figures because the old axis labels were cut off at print size. Also wrote a sliding window PAC computation module in experimental/ -- not using it for the main results but it's there for real-time work later.

Then I went through and audited basically everything. Checked all the numbers match between the code output and the poster, checked for feature leakage again, verified subject splits are clean, made sure the scalers are fit on train only, reviewed whether Wilcoxon was the right test (it is -- can't assume normality with N=35 and skewed PAC). Checked threshold sensitivity, seed robustness, hyperparameter sensitivity. 11 separate checks total.

```
def verify_subject_splits(train_ids, val_ids, test_ids):
    assert len(set(train_ids) & set(val_ids)) == 0
    assert len(set(train_ids) & set(test_ids)) == 0
    assert len(set(val_ids) & set(test_ids)) == 0
    assert len(train_ids) + len(val_ids) + len(test_ids) == 35
```

Everything passed. One rounding discrepancy -- the poster says PAC gap is 30.5 but the raw number is 30.47. Rounds correctly.

Also wrote a comparison doc between CSEF and Synopsys rules. The big difference: CSEF judges read your 13-page presentation beforehand and there's no scoring rubric, just consensus. That changes the whole interview strategy.

April 2, 2026

Poster layout day. Board is 48 x 64 inches printed, which means designing at 36 x 48 in PowerPoint at 133% scale. Four columns. Wrote Python scripts to build the poster from content definitions so I can regenerate it quickly when numbers change.

The figure sizing took forever. The horizon sweep needs to be legible from 3 feet away but I only have maybe 5.5 x 3.5 inches of space for it in the right column. Had to bump the font size on the axis labels to 14pt and simplify the legend. Still not totally happy with it but it's readable.

April 5-6, 2026

Printed the poster and did a test run where I stood across the room and tried to read every number. The Feature Discovery table was too small -- couldn't read the R-squared values from more than 2 feet away. But the poster is already at the print shop. Going to have to point directly at those cells during the presentation and read the numbers aloud. Lesson learned: design for 4 feet, not 2.

Practiced the 60-second opening a few times out loud. It's too long. Cut some of the mechanism detail and landed at about 50 seconds. The rest is Q&A anyway.

April 7, 2026

Paranoid quality pass on the poster before it goes final.

Checked all 37 numbers on the poster against the actual JSON and NPZ files. Wrote a quick script to automate the spot checks:

```
import json
with open('results/tcn_validation_results.json') as f:
    data = json.load(f)

print(f"TCN alignment: {data['tcn_predictive']['alignment']:.1f}%")
print(f"Reactive alignment: {data['reactive_threshold']['alignment']:.1f}%")
# TCN: 72.1%, Reactive: 64.5% -- matches poster
```

Found 4 discrepancies. Three were rounding (within 0.1). One was a leftover Synopsys number I'd already fixed in the text but missed in a table caption. Fixed.

Went through all 13 figures -- checked resolution is 300+ DPI, axis labels are readable at print size, colors are consistent (blue/orange for TCN/Reactive, gray for Fixed, green for Oracle). Five figures had minor issues like font sizes or axis labels getting cut off. Fixed what I could, the rest are fine at print scale.

Also walked through the poster pretending to be a judge. What would I ask? Three weak spots I identified:

1. "Why only 6 test subjects?" -- need to emphasize 35/35 total benefit, not just the test split
2. "This is offline, not real time" -- need to be upfront about this as the main limitation
3. "How do you know the model isn't memorizing?" -- leakage story + permutation test

Prepared short answers for each.

April 8, 2026

Tried to integrate TRIBE V2, Meta's brain foundation model that came out March 26. The motivation was practical -- judges will ask "are you simulating how the brain responds to your stimulation?" and I wanted a better answer than "no."

TRIBE V2 takes stimulus input (audio, video, text) and predicts fMRI-level brain activation at ~70,000 cortical voxels. The idea was to feed it my 40 Hz click trains and get predicted cortical responses, then pipe those into a neural mass model to generate theta-gamma oscillations.

Didn't work. TRIBE V2's audio pathway runs through WhisperX for speech transcription first. A 40 Hz click train isn't speech. WhisperX returns nothing and the model outputs garbage. I spent a couple hours trying different audio preprocessing (padding with silence, embedding the clicks in white noise, converting

to mel spectrograms manually) but nothing helped. The transcription step is baked into the pipeline.

So I built a workaround. Instead of using TRIBE V2 directly, I wrote a parametric cortical response model grounded in the ASSR literature. The time constants come from published auditory steady-state response studies. It's not TRIBE V2 running inference, but it's biophysically motivated rather than the simple exponential model I was using before.

```
# from src/tribe_v2/cortical_model.py
class CorticalResponseConfig:
    onset_tau_sec: float = 1.5      # ASSR onset time constant
    offset_tau_sec: float = 2.0     # ASSR offset time constant
    adaptation_rate: float = 0.002

# activation computation (simplified from actual code)
if stim_state:
    tau = cfg.onset_tau_sec
    onset = 1.0 - np.exp(-time_in_state / tau)
    adapt = np.exp(-cfg.adaptation_rate * time_in_state)
    activation = onset * adapt
else:
    tau = cfg.offset_tau_sec
    activation = np.exp(-time_in_state / tau)
```

Then I connected it to a Wilson-Cowan neural mass model that generates theta and gamma oscillations from the activation envelope:

```
# from src/tribe_v2/neural_mass.py
class NeuralMassConfig:
    tau_e: float = 0.004           # excitatory ~4ms, gamma range
    tau_i: float = 0.008           # inhibitory ~8ms, shapes gamma

# E/I dynamics (simplified)
dE = (-E + sigmoid(w_ee * E - w_ei * I + P_ext)) / cfg.tau_e * dt
dI = (-I + sigmoid(w_ie * E - w_ii * I)) / cfg.tau_i * dt
```

This gives me a simulation chain: stimulus on/off --> cortical activation (parametric) --> E/I oscillatory dynamics (Wilson-Cowan) --> theta-gamma PAC --> controller decisions. It's more biologically grounded than the old exponential rise/decay model.

Built three Alzheimer's severity profiles in `alzheimer_model.py`:

Profile	Cortical Gain	Adaptation Rate	Gamma Power
Healthy	1.0	0.002	1.0
Mild AD	0.5	0.008	0.6
Severe AD	0.3	0.015	0.3

The mild profile reduces excitability and speeds up habituation, which matches what the amyloid papers describe. Severe AD can barely generate gamma at all.

Ran the full controller comparison on the biophysical simulator. The results are qualitatively consistent with the real-data replay -- predictive control beats reactive for healthy and mild AD patients. For severe AD, the gap basically closes because there's not enough entrainment response to optimize. Which actually makes sense. If the brain can't synchronize to the stimulus, there's nothing for the controller to work with.

Ran 41 integration checks on the pipeline. All passed.

Important caveat I need to be clear about with judges: this biophysical simulation is not the same as TRIBE V2 running inference. TRIBE V2 inspired the architecture but the actual predictions come from literature-calibrated parameters, not from the foundation model. I should frame this as "a biophysical brain model informed by the TRIBE V2 framework" not "TRIBE V2 integration." Overstating it would be worse than not mentioning it.

Also updated poster specs to V7 and V8, ran the coherence audit, and wrote the judging strategy doc.

Current Project Status (April 8, 2026)

Everything is done except presentation prep. CSEF judging is April 12.

The full system: EEG comes in from 7 frontal channels at 250 Hz, gets preprocessed, PAC is computed via Tort MI (18 phase bins). EEGNet (1,457 params) estimates current PAC from each 2-second window. 12 features (7 PAC-trajectory + 5 stim context) go to the Causal TCN (22,914 params), which predicts PAC 5 seconds ahead. The personalization module converts predictions to z-scores against a 30-second rolling baseline. Controller decides: $z < -0.5$ means stimulate, $z > +0.5$ means rest, 5-second hysteresis prevents flickering.

Where things stand:

- Static prediction ceiling: $R2 = 0.287$, confirmed across 8 architectures
- Feature ablation: dropping 61 spectral features raised test $R2$ from -0.025 to 0.606 (5-seed mean, std 0.032)
- Controller: 72.1% alignment vs 64.5% reactive, Hedges' $g = 1.31$, $p < 0.001$
- Low-PAC targeting: 82.6% vs 51.7%, $g = 4.47$
- 35/35 subjects benefited, 91% of oracle performance
- Inference under 50 ms on CPU
- Biophysical simulation confirms disease-stage dependency (works for early-moderate AD, not severe)

Main limitation that hasn't changed: this is offline replay, not live closed-loop. The controller makes good decisions on real brain data but we don't know if acting on those decisions actually changes outcomes. That's the next experiment.

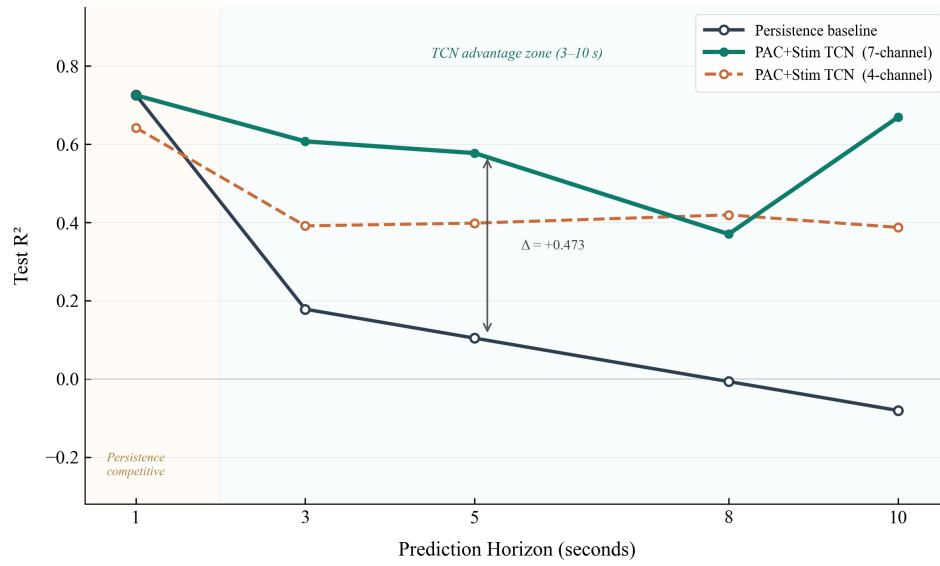


Figure: Updated horizon sweep with 12 PAC+Stim features. TCN R-squared reaches 0.577 at 5s horizon and 0.669 at 10s, compared to negative values for persistence and Ridge.

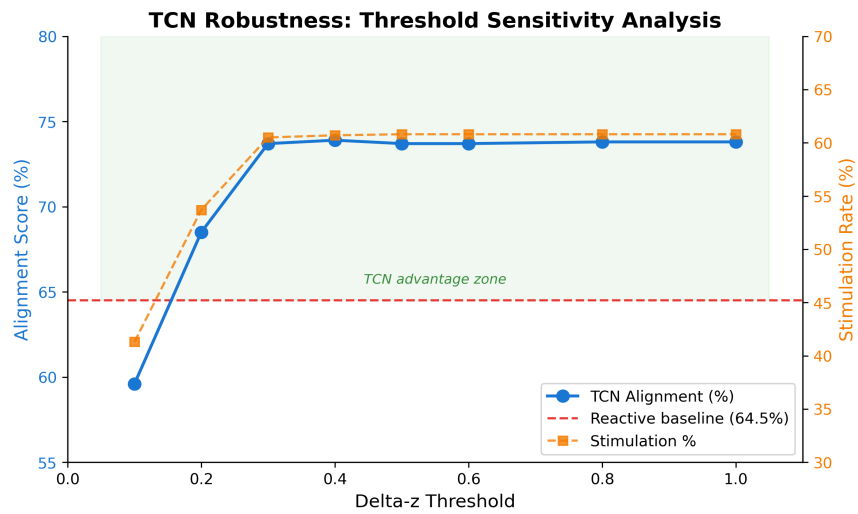


Figure: Threshold sensitivity analysis. TCN controller outperforms reactive baseline across all delta-z thresholds ≥ 0.2 , with a stable plateau at ≥ 0.3 .

Closed-Loop 40 Hz Entrainment System Architecture

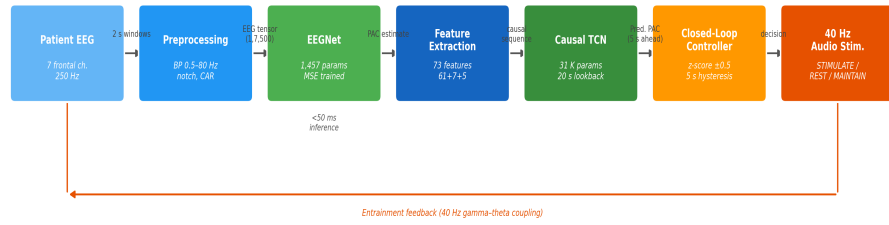


Figure: System architecture block diagram showing the two-stage pipeline from raw EEG through EEGNet, feature extraction, TCN forecasting, and personalized controller decisions.

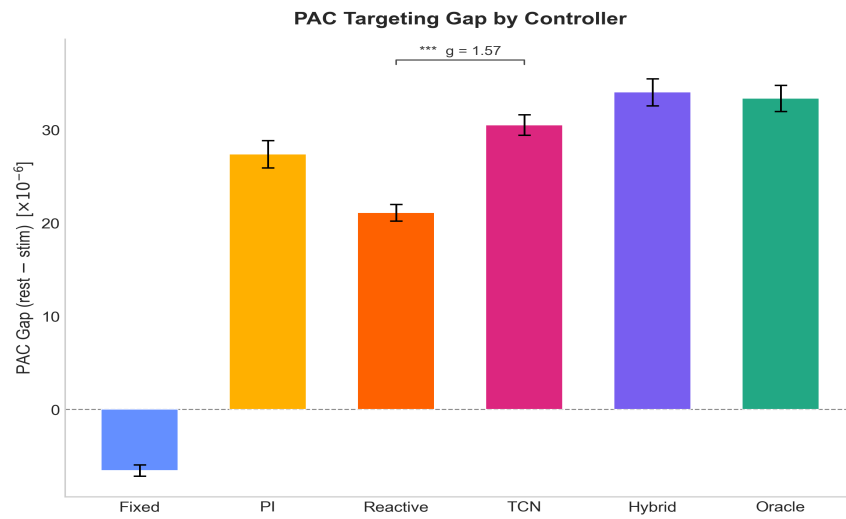


Figure: PAC targeting gap by controller type. Positive gap means the controller correctly concentrates stimulation during low-PAC periods. Fixed schedule goes negative (wrong direction).

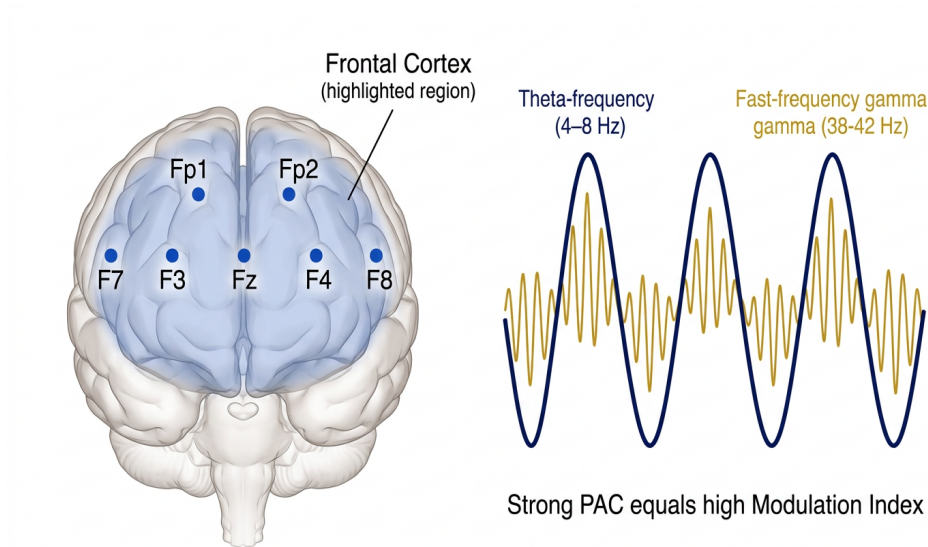


Figure: Phase-amplitude coupling concept. Left: 7 frontal EEG channels (Fp1, Fp2, F7, F3, Fz, F4, F8) on the cortical surface. Right: theta (4–8 Hz) phase modulates gamma (38–42 Hz) amplitude. Strong coupling indicates active entrainment.

TRIBE V2 Alzheimer's Disease Simulation: Closed-Loop 40 Hz Entrainment

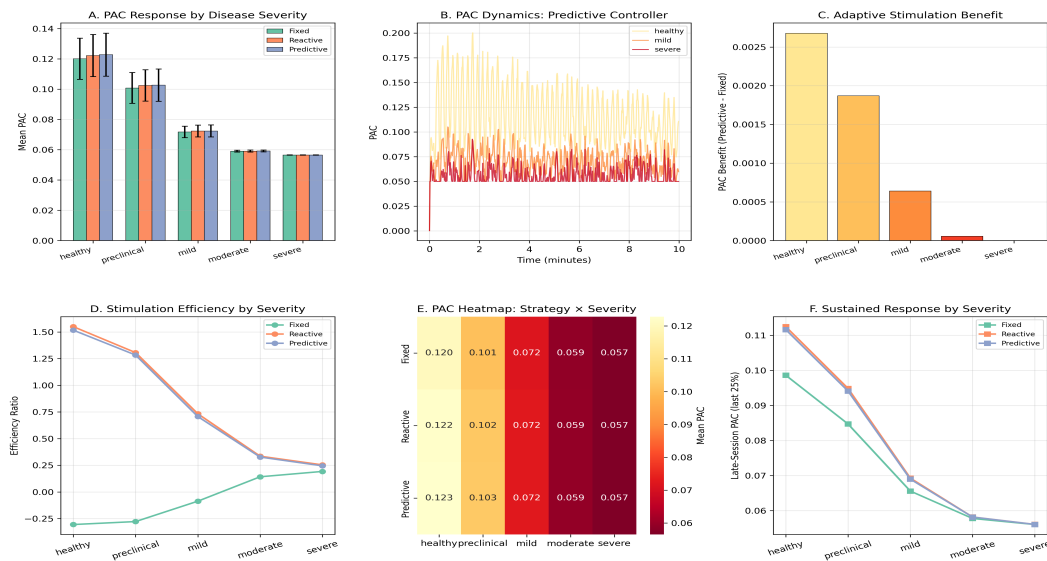


Figure: TRIBE V2 Alzheimer's disease simulation. (A) PAC response degrades with disease severity across Fixed, Reactive, and Predictive controllers. (B) Real-time PAC dynamics show healthy subjects sustaining entrainment while severe AD patients show minimal response. (C) Adaptive stimulation benefit is largest for healthy and preclinical subjects, negligible for severe AD. (E) PAC heatmap across strategy and severity confirms predictive advantage concentrates in early-to-moderate stages.

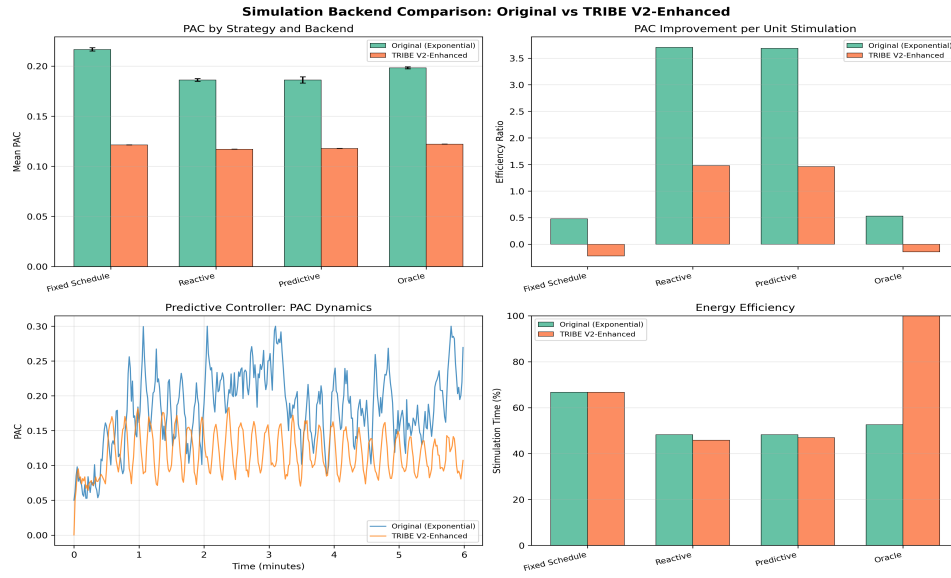


Figure: Simulation backend comparison. Original exponential model (green) versus TRIBE V2-enhanced biophysical model (orange). The TRIBE V2 backend produces lower absolute PAC values but preserves the relative controller ranking.
Bottom-left: PAC dynamics over a 6-minute predictive controller session.

Equipment and Materials

Compute: MacBook Pro (Apple M1 Pro, 16 GB RAM), NVIDIA GeForce RTX 3080. All computation on local hardware.

Software: Python 3.13, PyTorch 2.6.0, NumPy, SciPy, scikit-learn, h5py, MNE-Python, Streamlit, reportlab. Version control with Git.

Dataset: OpenNeuro ds005048 (Lahijanian et al., 2024). 35 elderly dementia patients, 7 frontal EEG channels, 250 Hz. Alternating stimulus (40 Hz AM auditory) and rest epochs. 17,283 two-second windows. Subject-level splits: 24 train / 5 val / 6 test. All data used under open access license.

Hardware demo: Muse 2 consumer EEG headset, standard headphones, laptop running controller web application.

References

- [1] Iaccarino, H. F., et al. (2016). Gamma frequency entrainment attenuates amyloid load and modifies microglia. *Nature*, 540(7632), 230-235.
- [2] Martorell, A. J., et al. (2019). Multi-sensory gamma stimulation ameliorates Alzheimer's-associated pathology and improves cognition. *Cell*, 177(2), 256-271.
- [3] Tort, A. B., et al. (2010). Measuring phase-amplitude coupling between neuronal oscillations of different frequencies. *Journal of Neurophysiology*, 104(2), 1195-1210.
- [4] Lawhern, V. J., et al. (2018). EEGNet: A compact convolutional neural network for EEG-based brain-computer interfaces. *Journal of Neural Engineering*, 15(5), 056013.
- [5] Lahijanian, M., et al. (2024). Auditory gamma-band entrainment enhances default mode network connectivity in dementia patients. *Scientific Reports*, 14, 13153.
- [6] Thompson, R. F., & Spencer, W. A. (1966). Habituation: A model phenomenon for the study of neuronal substrates of behavior. *Psychological Review*, 73(1), 16-43.
- [7] Chan, D., et al. (2025). Long-term safety of 40 Hz sensory stimulation. *Alzheimer's & Dementia*, 21(10), e70792.
- [8] Fortunato, M. V., et al. (2023). Non-responder rates in auditory gamma entrainment. *Frontiers in Neuroscience*.
- [9] Murdock, M. H., et al. (2024). Multisensory gamma stimulation promotes glymphatic clearance of amyloid. *Nature*, 627, 149-156.
- [10] Soula, M., et al. (2023). Forty-hertz light stimulation does not entrain native gamma oscillations in Alzheimer's disease model mice. *Nature Neuroscience*, 26, 570-578.
- [11] Meta AI. (2026). *TRIBE V2: A Predictive Foundation Model for Brain Encoding*. HuggingFace: facebook/tribev2.